

Mémo création carte Leaflet

Table des matières

1	vershtml.....	2
1.1	Arguments	2
1.2	Génération du source html	3
1.3	versgeojson.....	3
2	Création carte Leaflet.....	4
2.1	Colonnes spécifiques des vues 4Map.....	4
2.2	Afficher/masquer (griser) éléments de la légende à ouverture.....	4
2.3	Ordre des couches et libellés légende.....	5
2.4	Priorités affichage points, lignes, polygones sur la carte	6
3	Debug	7
3.1	Origines des bugs	7
3.2	Types de bugs et action associée	7
3.3	Alertes auto.....	8
4	Exemples de génération	9
4.1	Lancement ps1.....	9
4.2	Lancement pgadmin	9
4.3	Vue 4Map.....	9

1 vershtml

1.1 Arguments

Fonction postgres générant la carte Leaflet html

6 arguments dont 5 optionnels avec des valeurs par défaut

Renseigner obligatoirement ‘_nom_vue’, format tableau de texte

Arguments				
	Type de données	Mode	Nom de l'argument	Valeur par défaut
	text[] v	IN v	_nom_vue	
	text v	IN v	_caseacoher	'0':text
	text v	IN v	_titre	'Carte DGAML':text
	boolean v	IN v	_clic_zoom_actif	true
	text v	IN v	_info_id	'vue_defaut':text
	text v	IN v	_rech_json	'false':text

- **_nom_vue** : tableau de vues, syntaxe SQL : vershtml_n (array[‘vue1’,‘vue2’, ...])
- Si **_caseacoher** n’est pas renseigné aucune couche n’est cochée à l’ouverture (‘0’ par défaut). Si ‘1’ => 1ere couche cochée et pas les autres car 0 par défaut, ‘1,1’ : 2 1eres couches cochées, ‘1,0,1’ : 1^{er} et 3eme couches cochées. Si 5 couches présentes et qu’on veut cocher juste la 5eme, mettre : ‘0,0,0,0,1’. Cela concerne les cases à cocher seulement pas les éléments affichés via légende à l’ouverture qui est activé via la col. ‘afficherlegouverture’ voir + bas (*)
- **_info_id** = valeur de la col ‘id’ de la table ‘infos_carte’ qui indique au code js de remplir le bloc Info par le contenu de la col. ‘texte_info’. NB : la f° ps1 ‘SI3P0-Ajouter-Information-Carte’remplit la table avec le texte info inclus dans le .ps1
- **_rech_json** est composé de 3 valeurs séparées par des « | »:
 1. n° couche à traiter = ordre de passage dans array[‘vue1’, ‘vue2’, ‘vue3’, ...]
 2. filtre(s) de recherche = champ ‘leafletsearch’ ou nom de champ unique
 3. texte de l’invite de saisie, décrit l’objet recherché et les filtres

On peut mettre **plusieurs box** de recherche en séparant par des « ; ». Ex.: -
rechJson '1|leafletsearch|TMJA par RD, Année ; 4|RD |RD par n°'

Si choix de recherche sur 1 col., respecter espaces du nom de col. Ex. ‘Nom ‘. Col. ‘leafletsearch’ de la vue concatène plusieurs champs servant de filtre de recherche (exemple : ‘id+nom+adresse’ as leafletsearch). Adapter si besoin, le tri de la liste trouvée lors de la création de la col. leafletsearch (vérifier avant si besoin l’impact sur le tri existant de la vue)

(*) selon les manip il peut donc y avoir des éléments affichés 2 fois sur la carte. Veiller à ne pas afficher des doublons dès l'ouverture via activation case à cocher et éléments légende.

1.2 Génération du source html

Ligne de génération complète en fin de f° vershtml_n :

'return partieDébut || versgeojson_n (_nom_vue) || partieCodeJS'

- 1ere partie : appels aux api js, css et déclaration de variables globales
- 2 eme partie : couches geojson stockées dans un tableau js
- 3 eme partie : cœur du code js : préparation données geojson, création carte, ajout de fonctionnalités (fonctions js dédiées)

Pour la 1ere partie : l'ensemble des fichiers js et css (api leaflet, plugins tiers) référencés sont stockés en local. Cela évite les bugs suite à changements/suppression des fichiers en ligne et permet de les modifier pour les adapter aux besoins Dgaml. En contrepartie on ne bénéficie pas d'éventuelles améliorations => à vérifier ponctuellement en ligne. Ex : v. beta 2.0 Leaflet au 04/26.

1.3 versgeojson

La fonction vershtml_n() appelle une autre f° versgeojson_n() qui génère les couches geojson issues des vues passées en param dans '_nom_vue' (tableau de vues).

- Conversion auto de tout format L93 en WGS84.
- Lignes et polygones transformés en ST_MultiLineString et ST_MultiPolygon
- Couches 2D/3D (3 coord x,y,z) passées en 2D via f° 'ST_Force2D'
- Lignes classées par ordre de grandeur décroissantes pour que les tronçons les plus petits soient en dernier dans le geojson donc dessinés en dernier, donc cliquables (et surlignables via switch) car situés au-dessus des plus grands

2 Création carte Leaflet

Création de vues basées sur les tables métiers du schéma « m ».

2.1 Colonnes spécifiques des vues 4Map

- Chaque vue doit obligatoirement contenir une colonne '**geom**' contenant des coord. S'il y a des éléments de couche sans coord ils sont ignorés mais la carte affichera quand même les autres éléments renseignés
- Pour éviter une erreur à la génération, le js detecte les vues vides (0 ligne ramenée) ou sans aucunes coordonnées et les renomme en 'Couche x vide'
- Le nom de la couche dans le bloc case à cocher vient de la col. '**nomcouche**'. Si non renseigné, le js nomme en 'Couche 1', 'Couche 2' etc. selon ordre passage. Eviter variables dans nom sinon risque d'avoir 2 ou + valeurs distinctes
- Pour les couches lignes, ajouter et remplir une colonne '**couleur**' dans la vue. Si colonne absente, couleur par défaut = rouge
- Pour les couches points, ajouter et remplir une colonne '**icone**' dans la vue. Valeur : url vers l'icone. Ex '/Ressources/Images/Punaises/pr.png'. Si col. icone absente, icone par défaut = punaise bleue
- Pour ajouter une légende, ajouter et remplir une colonne '**legende**' dans la vue. Les valeurs sont issues d'1 ou plusieurs col. métier ou définies en dur ou mixtes
- Les autres col. de la vue formeront le contenu des popups Leaflet. Les champs sont ordonnés dans la fenêtre par longueur de nom de champ croissante => jouer sur les espaces au nommage pour les classer selon besoin

Traitement js spécifique des champs et valeurs dans les popups :

- couleur|legende|icone|nomcouche|afficherlegouverture|leafletsearch exclus
- Valeurs de champs vides de la vue mis à « Non défini »
- Champs commençant par « Lien » sont formatés en liens http(s) actifs et exclus des infobulles (inutiles car cliquables depuis 1 infobulle).

2.2 Afficher/masquer (griser) éléments de la légende à ouverture

Par rapport aux cases à cocher, la légende a une granularité + grande en affichant/masquant les éléments d'une même couche => nécessite une vue adaptée

- Créer col. 'legende' contenant les libellés des légendes. A chaque libellé légende distinct doit correspondre un icone ou couleur spécifique.
- Par défaut, avec 1 col. 'legende' et 'icone' ou 'couleur' renseignés, les éléments seront actifs dans la légende à ouverture et affichés sur la carte

- Pour griser tous les éléments d'une couche contenant une col. legende à l'ouverture, ajouter 1 col. '**AfficherLegOuverture**' et la mettre à 'false' ou la laisser vide. On peut réactiver par la suite, en mettant sa valeur à 'true'
- Pour griser une partie des éléments d'une couche, créer des filtres sql. Ex.
 - "CodeVigilanceHydro > 1 as AfficherLegOuverture" : type alerte >1 actives dans légende, celles = 1 grisées
 - "case when couleur = '#F29120' or couleur = '#449D67' then false else true end as AfficherLegOuverture" : 2 éléments spécifiques de la couche grisés dans légende à ouverture, les autres activés (else true). On aurait pu référencer col.' legende' à la place de 'couleur' pour même résultat

2.3 Ordre des couches et libellés légende

Respecter ordre des vues selon type géométrique : **points, lignes, polygones** *sinon décalage au clic légende*. Les légendes seront donc aussi classées par type geom.

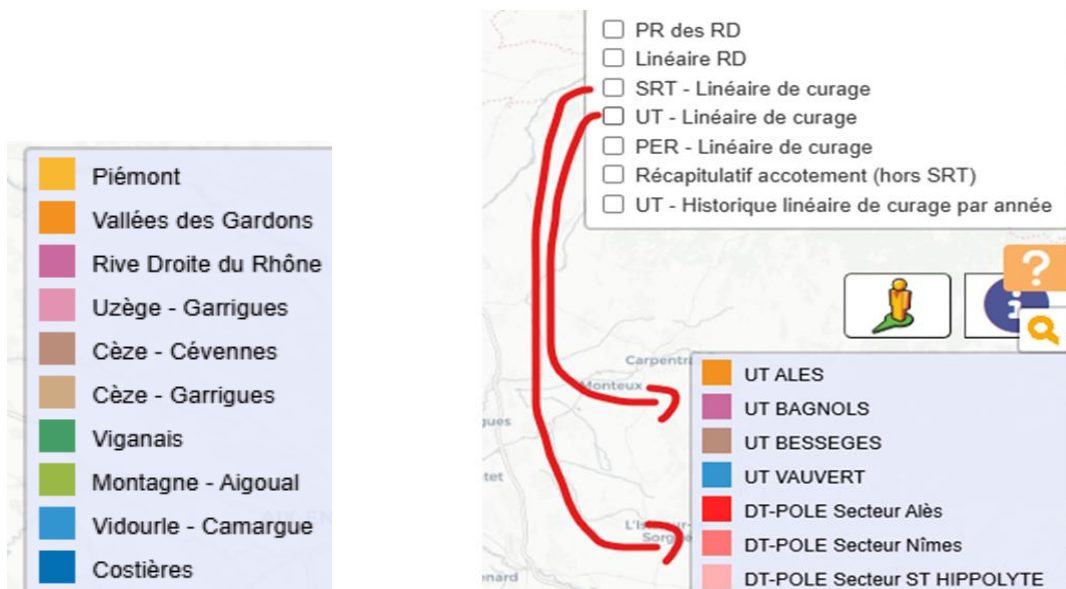
Les libellés légende de couches de même type geom suivent l'ordre de passage des vues/couches du bloc case à cocher => libellés couche 1 sont inscrits en 1^{er} dans la légende donc poussés vers le bas par les libellés des couches suivantes (pile, cf. schéma curage SRT/UT). *voir si besoin de changer décalage couche/légende*

Les libellés légende au sein d'une couche point ou polygone suivent l'ordre des lignes défini dans la vue (ordre alpha si aucun tri). Exemple : 'couleur' alimentée dans un ordre correspondant aux ST d'une UT en SQL

Les libellés légendes au sein d'une couche lignes suivent le order by des tronçons desc de versgeojson remplaçant l'ordre initial défini dans la vue => js modifié pour ordonner quand même les libellés lignes:

- ordre alphabétique intelligent des lignes (ex S1,S2, S10 au lieu de S1, S10,S2)
- éléments grisés placés après les actifs et classés aussi par ordre alphabétique

NB : Avec le tri alpha., utiliser des astuces de nommage de libellé pour avoir l'ordre souhaité. Exemple: '1 – Niv. Structurant', '2 – Niv. de liaison', '3 – Niv. de proximité'



Polyg., ordre lignes couleur

Lignes : ordre arrivée couches + ordre alpha. même couche

Les doublons couleur ou icône sont permis si libellés légende distincts. Eviter les doublons 'couleur/icône - libellés légende' (fusionnés dans légende).

Les vues avec 1 col. légende mais sans col. icône/couleur ne s'afficheront pas. Si on crée une légende, toujours attribuer 1 col. icône/couleur.

2.4 Priorités affichage points, lignes, polygones sur la carte

- Priorité entre types geom à l'ouverture et en cours de consultation:
 - le js priorise toujours l'affichage des points (icônes) sur lignes/polygones
 - le js priorise toujours les lignes sur polygones. Ainsi lignes restent toujours cliquables sous un polygone même s'il vient d'être activé
- Priorité entre lignes :
 - à ouverture, le dernier tronçon dessiné (du + grand au + petit cf. versgeojson) s'affiche par dessus les autres lignes
 - en consultation, le dernier élément ligne de la légende ou la dernière couche ligne activée s'affiche par dessus les autres lignes.
- Priorité entre points :
 - à ouverture, le 1er élément légende s'affiche par-dessus les autres
 - en consultation, le dernier élément de la légende ou la dernière couche point activée s'affiche par-dessus les autres icônes

3 Debug

Tester toute modification du source d'abord en dur et à part sur le serveur [IIS qualif](#). Si des modifs de vershtml sont nécessaires, tester toute non régression avant.

3.1 Origines des bugs

- suite à modif ou ajout de fonctionnalités dans vershtml pas ou mal testées
- suite à oubli ou mauvaise alimentation (valeurs, formats, doublons) des colonnes geom et spécifiques des vues 4Map : geom, nomcouche, légende, ...
- suite à traitement de couches geojson contenant des valeurs ou incohérences jamais rencontrées jusque-là (*)
- autres : syntaxe script ps1 KO, modifs/perturbations sur SI CD30, SI tiers, etc.

(*) Le contenu 'anormal' d'une couche geojson s'explique par de nombreux facteurs : modifs effectuées via f° postgis avant transfo en geojson (phase alim), erreur/chgt de saisie à la source, évolution logicielle/API/formats, évolution du référentiel, des règlements... cette évolutivité doit aussi s'anticiper : prise d'info, newsletter, bot IA

3.2 Types de bugs et action associée

Carte ne se mettant pas ou plus à jour => après analyse des rapports d'erreur ps1:

- script sql de l'une des vues 4Map buggé => debug sous pgadmin
- script lancement ps1 buggé : attention aux caractères, tabulations de trop, retours lignes, C/C entre lignes de lancements mal adaptés

Carte plantée à ouverture ou après manipulations :

- Faire F12 console dans Edge pour identifier bug et localisation (n° ligne). Rajouter ou décommenter des consoles.log, s'aider des commentaires js.
- Si non expert js, interroger la vue 4Map sous pgadmin ou isoler les couches geojson en début de code et analyser leur contenu sur des mots clés : "coordinates", "type", "nomcouche", "legende", etc.

Une vue/couche ne s'alimente plus (ou n'a plus de coord):

- la couche est quand même indiquée en tant que 'couche n vide' dans box
- identifier l'origine du changement : source vide ou accès KO aux fichiers, logiciels, sites, API, .ps1/sql 'moissonner' d'alim. KO suite chgt formats, etc.

Le plantage d'une carte a « l'avantage » de soulever une anomalie sur les données reçues et d'identifier après analyse, les données à corriger. **Attention** : un filtre SQL sur 1 couche geojson va débbugger la carte mais aussi tronquer les infos visibles dessus => **corriger à la source en priorité**. Si impossible et qu'un patch SQL ou js est

appliqué, vérifier au cours du temps, la cohérence des infos visibles sur carte ⇔ sources de données (tables taradata, autres)/contenu vues et tableaux si3p0 liés.

3.3 Alertes auto

Prévoir un système d'alertes automatisées type envoi auto de mails (en ps1 ou python) sur erreurs repérables :

- cartes dyn. à 0 ko après script quotidien
- cartes dyn. non mise à jour depuis x jours
- parsing des rapports d'erreur .txt et .err pour relevé d'anomalies par rapport à des mots clés type 'rollback'
- lien/récupération des alertes auto de teradata (API?)
- Autre

4 Exemples de génération

4.1 Lancement ps1

Les noms des paramètres ps1 diffèrent légèrement des noms des arguments sql mais sollicitent bien leur équivalent pour générer une carte.

Ajout du paramètre '-carte' pour indiquer nom et chemin de génération de la carte

```
[void]$parametresJobs.Add((  
    Parametrer-Job-SI3P0-Generer-Carte `  
        -sources 'tmja_mensuel_global_4Map', 'ReseauRoutier_4Map', 'LimiteST_4Map'  
        -couchesActives 'LimiteST_4Map' `  
        -carte "$si3p0ThematiquesPortailWeb\Exploitation et trafic routier\Cartes  
        dynamiques\Trafic moyen journalier annuel (TMJA).html" `  
        -titre 'Trafic moyen journalier annuel (TMJA)' `  
        -idInfo 'MJA5AnsComptageRoutier' `  
        -rechJson '1|leafletsearch|TMJA par RD, Année, Commune, Type ; 2|RD |RD' `))
```

Attention : si on ajoute 1 paramètre de lancement, matcher l'ordre des paramètres de 'Parametrer-Job-SI3P0-Generer-Carte' ⇔ 'vershtml_n()' (contrainte ps1)

Si ps1 abandonné, créer l'équivalent de la f° « SI3P0-Generer-Carte » dans le langage concerné (python...). Recréer la connexion API à la base et le mapping des arguments.

4.2 Lancement pgadmin

Ordre de passage des arguments libres

```
select vershtml_n (array['structuresrh', 'entretien_utsal', 'limitegestion_ut'],  
_caseacoher := '0,0,1', _rech_json := '1|Nom |structure RH par nom ;  
2|leafletsearch|activité par RD, prad, date', _titre := 'test – fauchage UT Ales' ;
```

4.3 Vue 4Map

- 2 couches créées selon valeurs date récente/ancienne (cas particulier pour cette vue).
- Alertes waze grisées à ouverture dans la légende sauf accident, danger véhicule, inondation.
- Icones personnalisés attribués dynamiquement selon type d'alerte

```

create view AlerteWaze_4Map as
select
    case when (Age(Now(), DateCreation) < interval '6 hours') then 'Alerte récente Waze' else 'Alerte ancienne Waze' end as "Nature ",
    to_char(a.DateCreation, 'DD/MM/YYYY HH24:mi') as "Date de création ",
    p._NumeroRoute as "RD",
    PRAEnTexte(p._PRA) as "PR+Abs",
    t.Description as "Type",
    st.Description as "Sous-type",
    a.Fiabilite as "Fiabilité",
    'https://www.waze.com/live-map?lat=' || round(ST Y(TransformerEnWGS84(a.Geom))::numeric, 6) || '&lon=' || round(ST X(TransformerEnWGS84(a.Geom))::numeric, 6) as "Lien LiveMap",
    case when (Age(Now(), DateCreation) < interval '6 hours') then 'Alertes récentes Waze' else 'Alertes anciennes Waze' end as NomCouche,
    CASE
        WHEN t.description LIKE '%Accident%'
            OR st.description LIKE '%Accident%'
            OR st.description IN ('Danger présence véhicule urgence', 'Inondations') THEN true
        ELSE false
    END AS AfficherLegOuverture,
    case
        when (Age(Now(), DateCreation) < interval '6 hours') and a.IdSousTypeAlerteWaze <> '' then '/Ressources/Images/Waze/70x70/couleur/' || a.IdSousTypeAlerteWaze || '.png'::varchar
        when (Age(Now(), DateCreation) < interval '6 hours') and a.IdSousTypeAlerteWaze = '' then '/Ressources/Images/Waze/70x70/couleur/' || a.IdTypeAlerteWaze || '.png'::varchar
        when a.IdSousTypeAlerteWaze <> '' then '/Ressources/Images/Waze/70x70/gris/' || a.IdSousTypeAlerteWaze || '.png'::varchar
        when a.IdSousTypeAlerteWaze = '' then '/Ressources/Images/Waze/70x70/gris/' || a.IdTypeAlerteWaze || '.png'::varchar
        else '/Ressources/Images/Waze/70x70/couleur/MISC.png'::varchar
    end as Icone,
    case
        when a.IdSousTypeAlerteWaze <> '' then st.Description
        else t.Description
    end as Legende,
    a.Geom
from AlerteWaze a
cross join PointVersPRA(Geom, 25) p
inner join TypeAlerteWaze t on a.IdTypeAlerteWaze = t.IdTypeAlerteWaze
inner join TypeAlerteWaze st on a.IdSousTypeAlerteWaze = st.IdTypeAlerteWaze
where a.IdTypeAlerteWaze not in ('JAM', 'ROAD_CLOSED') and a.IdSousTypeAlerteWaze <> 'HAZARD_WEATHER_FOG'
and a.Fiabilite > 6;

```